

TWO LINUX SYSTEMS FROM SCRATCH

WHAT A LINUX SYSTEM IS, HOW IT BOOTS, AND HOW DISTRO/DISTROLESS EACH BUILD ONE

AUTHOR

Javier Fernández

CONTACT

jfernand@me.com

ISSUED

2026-09-15

ABSTRACT

This workspace builds two independent, bootable Linux systems entirely from upstream source, using nothing but the host's own compiler toolchain: `distroless` (musl + BusyBox + utils, minimal) and `distro` (glibc, from-scratch, aimed eventually at a COSMIC desktop). Part 1 explains what a Linux system fundamentally **is** — the bootloader/kernel/init layering every Linux system shares, and the handful of jobs (device management, seat/session, coreutils, graphics) every desktop-capable system needs done, one way or another. Part 2 explains how this workspace's shared buildpack architecture works, and then walks through each distro's own concrete build: what packages it compiles, in what order, and how each has been verified to actually boot.

DOCUMENT CONTROL

WORKSPACE	Cargo workspace: <code>buildpack-core</code> + <code>buildpacks</code> (shared package registry) · <code>builder-tui</code> (shared dashboard) · <code>distroless</code> (musl/BusyBox) · <code>distro</code> (glibc/from-scratch) · <code>distro-init</code> (PID 1)
TARGET	Native <code>x86_64-unknown-linux-gnu</code> — host toolchain, no cross-compilation
COVERAGE	Both distros' full pipelines, kernel through a booting, logged-in shell; <code>distro</code> additionally through Weston actually rendering a client via <code>virtio-gpu</code> inside QEMU
NOT COVERED	A Rust toolchain on <code>distro</code> 's own target, COSMIC itself, real GPU drivers beyond <code>virtio-gpu</code> , audio, networking UI — see §11

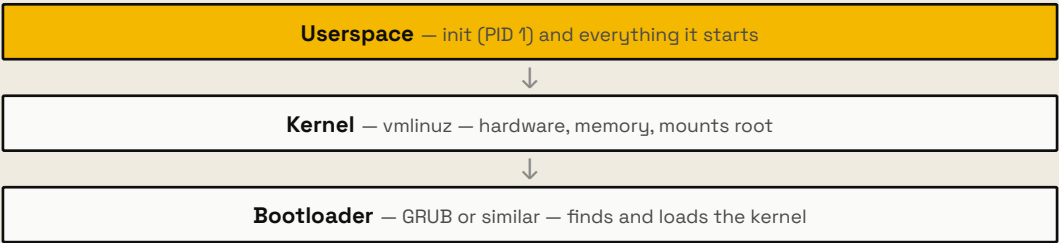
PART ONE

What a Linux system is, and how it boots

01

THE THREE LAYERS

Every Linux system, regardless of distribution, is the same three layers stacked on top of each other:



LAYER	JOB
Bootloader	Runs first, finds a kernel image on disk, loads it into memory, hands off control. GRUB is the common choice; there are others.
Kernel	One binary — <code>vmlinuz</code> . Initializes hardware, mounts the root filesystem, then execs exactly one user-space program as process ID 1. Everything the kernel does before that point (driver probing, <code>devtmpfs</code> , memory management) is the same regardless of what distribution eventually boots.
Userspace	Everything else — starting with PID 1 (<code>init</code>) and everything <code>init</code> , directly or indirectly, starts after it.

What actually distinguishes one Linux system from another lives almost entirely in that third layer: which `init`, which device manager, which `coreutils` implementation, which graphics stack, if any. A minimal embedded system and a full desktop share the exact same bootloader/kernel handoff — they differ in how much userspace machinery `init` brings up afterward.

02

HOW IT BOOTS, STEP BY STEP

STEP	COMPONENT	WHAT HAPPENS
1	Bootloader	Reads the kernel image off disk and hands off.
2	Kernel	Boots, mounts the root filesystem read-write, execs <code>/sbin/init</code> .
3	<code>init</code> (PID 1)	Mounts whatever else the running system needs(<code>/proc</code> , <code>/sys</code> , <code>/dev</code> , <code>/run</code>),starts every other userspace service, and from then on supervises them — if a required service dies, PID 1 is what notices and (usually) restarts it.
4	Device management	Started by <code>init</code> , early: makes the hardware <code>devtmpfs</code> already created nodes for actually usable — permissions, symlinks, a queryable “what’s plugged in” database.
5	<code>getty</code>	Opens a terminal (the console, a serial port, both) and prints a login prompt.
6	login	Reads a username and password, checks them against the system’s account database, and — if they’re valid — replaces itself with the user’s shell.

STEP	COMPONENT	WHAT HAPPENS
7	shell	An interactive command interpreter. The system is now usable.

Steps 1–2 are identical on every Linux system. Steps 3 onward are where a distribution’s own choices show up — how much init does directly, what it delegates to separate daemons, and what those daemons are.

2.1 FIRMWARE AND THE BOOTLOADER: GETTING TO STEP 1

A bootloader doesn’t run because the disk is bootable in some magical sense — the machine’s own firmware is what decides to run it, and **how** differs by firmware type:

FIRMWARE	HOW IT FINDS SOMETHING TO RUN
Legacy BIOS	Reads the first 512-byte sector of the boot disk (the MBR) and executes it directly as code — that sector is normally just a small stub whose entire job is to load a larger second stage from elsewhere on disk, since 512 bytes isn’t room for a real bootloader.
UEFI	Reads a <code>FAT32</code> -formatted partition specifically marked as the EFI System Partition (ESP) and executes a <code>.efi</code> file from it directly — a real PE-format executable, not a boot-sector stub. Which <code>.efi</code> file to run is normally recorded in the firmware’s own NVRAM as a named boot entry, or — if nothing is registered — UEFI falls back to a fixed, well-known path on the ESP: <code>\EFI\BOOT\BOOTX64.EFI</code> .

GRUB, install and all, is itself split the same way BIOS-era bootloaders were: a minimal loader the firmware hands off to, which then loads GRUB’s **real** logic — including reading `grub.cfg` — from wherever it was actually installed. Installing it with `--removable` specifically targets UEFI’s fallback path above rather than registering an NVRAM entry, so the resulting disk boots correctly on **any** UEFI firmware, unmodified NVRAM included — the same property that makes a “boot from USB” disk work on a machine that’s never seen it before.

`grub.cfg` itself does two things: `search --fs-uuid` locates a filesystem by its on-disk UUID and sets `$root` to wherever GRUB finds it (independent of `/dev/sdX` -style naming, which can differ between boots), and `linux` then loads the kernel image from that filesystem and sets its command line — critically, `root=PARTUUID=...`, a **different** identifier from the filesystem UUID `search` just used: the kernel itself only understands `PARTUUID=` natively for locating its own root partition at boot, not the filesystem UUID GRUB found it by.

2.2 TELLING WHERE YOU ACTUALLY BOOTED FROM

None of the above has to be taken on faith — a running system can be asked directly what booted it, from most to least specific:

CHECK	WHAT IT TELLS YOU
<code>cat /proc/cmdline</code>	The exact kernel command line GRUB set — including the literal <code>root=PARTUUID=...</code> (or <code>UUID=...</code>) value the kernel used to find its own root partition.
<code>findmnt /</code> or <code>cat /proc/mounts</code>	The actual device node (<code>/dev/vda2</code> , say) the kernel resolved that identifier to and mounted — after resolution, not the identifier itself.
<code>dmesg grep -i "mounted root"</code>	The kernel’s own boot-time log line recording which device and filesystem type it mounted as root — a permanent record of what happened, not just what was asked for.
<code>blkid</code>	Cross-references a <code>PARTUUID</code> / <code>UUID</code> from <code>/proc/cmdline</code> against every real device on the system, useful for going the other direction — “which physical partition does this identifier actually name?”
<code>[-d /sys/firmware/efi]</code>	Whether this boot went through UEFI at all — present only if it did, absent on a legacy BIOS boot. A different firmware path entirely (above), not just a detail of which partition was chosen.

2.3 FROM A LOADED IMAGE TO A RUNNING KERNEL

Handing off to the kernel isn't handing off to a fully capable OS yet — the image GRUB loaded is compressed, and unpacks itself first via a small stub linked into the front of the file. Once running for real, the kernel walks the memory map the firmware handed it, brings up its own core subsystems (scheduler, memory management, the device model underlying §3), and probes for hardware, loading whatever drivers are built directly into this kernel image as it finds matching devices.

Some systems need a userspace-driven step **before** any of this can even locate a real root filesystem — a RAID array to assemble, an encrypted volume to unlock, or a storage driver too specialized to be built into the kernel image itself, handled by a temporary **initramfs**: a small, self-contained root filesystem, embedded alongside the kernel image, whose only job is running just enough userspace to make the real root filesystem reachable before handing off to it and being discarded. A kernel built with every storage driver it needs compiled directly in — true for both `distro` and `distroless`, §9–10 — has no such gap to close, and mounts its real root filesystem directly, with no `initramfs` stage at all.

2.4 THE PSEUDO-FILESYSTEMS INIT MOUNTS, AND WHY

None of `/proc`, `/sys`, `/dev`, or `/run` hold real files on disk — each is a **pseudo-filesystem**, generated live by the kernel (or, for `/dev`, populated live as described in §3) rather than read from a block device, and each has to be mounted explicitly by `init` before anything depending on it can work:

MOUNT	WHAT IT ACTUALLY IS
<code>/proc</code>	A live view of kernel and per-process state — one directory per running PID, plus kernel-wide tunables and statistics. Standard tools (<code>ps</code> , <code>top</code> , and plenty of libraries) read straight from here rather than through some other API.
<code>/sys</code>	<code>sysfs</code> — the kernel's device/driver model, described in full in §3.1.
<code>/dev</code>	<code>devtmpfs</code> — where device nodes actually live, populated as drivers bind to hardware, refined by <code>udev</code> 's rules pass (§3).
<code>/run</code>	An ordinary <code>tmpfs</code> — real files, but backed by RAM, not disk, and empty again on every boot. Where sockets, PID files, and other runtime-only state belong precisely because nothing should expect it to survive a reboot.

2.5 WHAT INIT ACTUALLY DOES, AND WHY IT'S NEVER “NOTHING”

PID 1 is not an implementation detail. The kernel will only ever execute **one** program as PID 1, and that program inherits a genuinely special job: it's the ancestor of every other userspace process, and it's what the kernel hands orphaned child processes to for reaping. A system with no working `init` doesn't boot to a shell — it boots to a kernel panic the moment that one process exits or crashes.

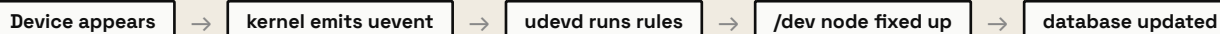
Three different answers to “what should `init` actually do” show up across real systems:

APPROACH	WHAT IT LOOKS LIKE
<code>systemd</code>	The default on most desktop distributions. Starts and supervises services in dependency order, described declaratively as “unit” files, alongside a large family of companion daemons (<code>logind</code> for sessions, <code>udev</code> for devices, <code>journald</code> for logging, <code>networkd</code> , and more) that a full <code>systemd</code> -based system typically runs together.
A small built-in <code>init</code>	<code>BusyBox</code> ships one: reads a short, static <code>/etc/inittab</code> , starts and respawns exactly the processes it names. No dependency graph, no unit files — just a fixed list.
A purpose-written <code>init</code>	Nothing says <code>init</code> has to be either of the above. A minimal <code>init</code> can be a short, direct program: mount what's needed, fork a fixed set of children, wait for any of them to exit, fork it again. §10.1.1's <code>distro-init</code> is exactly this — about 100 lines.

None of these is “more correct” than the others — they're different trade-offs between flexibility (`systemd` can express arbitrary service dependency graphs) and simplicity (a fixed list of children is easy to read in full and reason about completely).

03 DEVICE MANAGEMENT: UDEV

The kernel’s device drivers know about hardware the moment it’s detected, but “knowing about it” and “userspace can actually use it” are two different things. Getting from one to the other is a genuine pipeline, not a single step.



3.1 SYSFS: THE KERNEL’S HARDWARE MODEL, AS A FILESYSTEM

Every device, bus, and driver the kernel currently knows about is represented internally as a **object**, and the whole tree of them is exposed to userspace as `sysfs`, mounted at `/sys`. This is metadata and control, not device access: a directory per device, files inside it for that device’s attributes (`/sys/class/net/eth0/address`, say), and symlinks expressing the relationships between devices, drivers, and buses. Nothing in `/sys` is a device node you can `open()` and read bytes from — that’s a separate, second thing, over in `/dev`.

`devtmpfs`, mounted early in boot, is what actually populates `/dev`: as each driver binds to a device, the kernel itself creates the matching device node there (a special file recording a major/minor number pair, not real file content) with a kernel-chosen name and default, usually-too-permissive ownership. That’s the entire extent of what the kernel does on its own — correct device nodes exist, with root-only or otherwise generic permissions, no stable naming beyond whatever order drivers happened to probe in, and no way for anything to ask “what’s connected right now” after the fact.

3.2 UEVENTS: HOW THE KERNEL ANNOUNCES A CHANGE

Every time a device is added, removed, or changes state, the kernel emits a **uevent** — a small message, broadcast over a dedicated netlink socket (`NETLINK_KOBJECT_UEVENT`), carrying `KEY=value` pairs: `ACTION=add`, the device’s `/sys` path, its `SUBSYSTEM` (`input`, `block`, `drm`, ...), major/minor numbers, and a monotonically increasing sequence number so ordering survives even if events are handled out of strict arrival order. `udev` is a long-running process that does nothing but sit on that socket and react.

3.3 RULES: DECIDING WHAT A DEVICE ACTUALLY GETS

Reacting to a uevent means running it through **udev rules** — plain-text files (`/usr/lib/udev/rules.d/*.rules`, with `/etc/udev/rules.d` available to override them) matched top to bottom against that device’s properties:

DIRECTIVE	WHAT IT MATCHES OR DOES
<code>SUBSYSTEM==</code> , <code>KERNEL==</code>	Match against the uevent’s own subsystem and kernel-assigned device name.
<code>ATTR{name}==</code>	Match a <code>sysfs</code> attribute’s value — a device’s vendor/product ID, for instance.
<code>ENV{KEY}==</code>	Match a property attached by an earlier rule or a previous pass — rule evaluation is cumulative, not one-shot.
<code>SYMLINK+=</code>	Add a stable, descriptive symlink to the node <code>devtmpfs</code> already created — <code>/dev/disk/by-id/...</code> , <code>/dev/input/by-path/...</code> — so nothing has to hardcode a <code>sdx / eventN</code> name that can change between boots.
<code>MODE=</code> , <code>OWNER=</code> , <code>GROUP=</code>	Fix up the permissions <code>devtmpfs</code> ’s default was never going to get right for every device class.
<code>TAG+=</code>	Attach a label other tools query for later — <code>seatd</code> (§4) and <code>libinput</code> (§10.3.3) both rely on devices being tagged consistently to recognize what they are.
<code>RUN+=</code>	Run an external program as part of handling this event.

`udev` forks a worker process per device to evaluate its rules — in parallel across independent devices, but serialized for any one device so a rename can’t race a later rule that depends on the new name. Since

`devtmpfs` already created the node, most of what a rule does is refine it: rename or symlink it, correct its permissions, tag it — actual `mknod` only falls to `udev` itself on a system with no `devtmpfs` at all.

3.4 COLDPLUG VS. HOTPLUG

A `uevent` is a live, one-time broadcast — anything not yet listening when it fires simply never sees it. That’s exactly the situation at boot: `devtmpfs` creates nodes for every device already present **before** `udev` itself has started, so none of them ever produced a `uevent` `udev` was around to catch. **Coldplug** is the fix: `udevadm trigger` walks the current `/sys` tree and synthesizes a `uevent` for every device already there, run through the exact same rules as a live hotplug event. Skipping this step boots to a system where `/dev` nodes exist (courtesy of `devtmpfs`) but none of them have been renamed, symlinked, permission-corrected, or tagged — which is why `distro-init` (§10.1.1) runs `udevadm trigger` once, immediately after starting `udev`.

3.5 THE DEVICE DATABASE

Beyond the rules pass, `udev` maintains a live database under `/run/udev/data/`, one entry per device, holding everything a rules pass determined about it — its tags, its properties, its stable names. This is what `udevadm info --query=all --name=<path>` actually reads, and what lets other components ask “what kind of device is this” without re-deriving it themselves; `libinput` (§10.3.3) and `seatd`’s own device filtering (§4) both depend on this database being populated and current, not just on the raw `/dev` node existing.

Without something doing this whole job — `sysfs` walk, `uevent` listener, rules engine, database — a system boots to device nodes with no stable names, no correct permissions beyond whatever `devtmpfs` guessed, and no way to notice a device that appears after boot at all. `udev` is the canonical implementation; `eudev` is a fork that does the same job without depending on `systemd` (ordinary `udev` is a `systemd` subproject today) — the same rule syntax, the same `netlink`/`sysfs`/`database` mechanics, just packaged to build and run standalone. It’s the device- management answer this workspace actually uses (§9, §10).

04

SEAT & SESSION MANAGEMENT

A Wayland compositor needs exclusive, arbitrated access to raw input devices (`/dev/input/*`) and the GPU (`/dev/dri/*`) — but running it as root just to get that access is the wrong trade-off, and letting an unprivileged process `open()` those nodes directly hits real permission walls even if it wanted to. **Seat management** is the piece that solves this — and it turns out to bundle together several related problems, not just one.



4.1 WHAT A “SEAT” AND A “SESSION” ACTUALLY ARE

A **seat** is a bundle of hardware usable by one person at a time — at minimum a display, a keyboard, a pointer. Most machines have exactly one seat; a **multi-seat** system has several independent bundles (separate GPUs, separate input devices) so more than one person can use the same physical machine simultaneously, each with their own login. A **session** is the lifetime of one logged-in user’s occupancy of a seat — tracked as its own thing (not just “a process is running”) because a seat’s devices need to change hands cleanly: when a session stops being the foreground one, whatever was using its keyboard/GPU access needs to actually lose that access, not just keep holding a file descriptor open against a display no one can see.

4.2 WHY THIS NEEDS A BROKER AT ALL

Two problems, not one, have to be solved before a compositor can safely touch `/dev/input/*` and `/dev/dri/*` as an ordinary user:

PROBLEM	WHAT HAS TO HAPPEN
Permission	<code>/dev/input/*</code> and <code>/dev/dri/*</code> are root- or group-restricted by default (§3's <code>udev</code> rules decide exactly how). An unprivileged compositor process has no path to opening them directly.
Arbitration	Switching virtual terminals (<code>Ctrl + Alt + F2</code> , say) has to revoke the outgoing session's device access and hand it to the incoming one — two compositors racing for the same GPU is a crash, not a feature. The kernel's own VT subsystem signals this switch; something has to react to it and actually pause/resume the affected session's access.

A seat daemon solves both at once: it holds the real, privileged file descriptors open itself, and hands duplicated, access-controlled descriptors to whichever session currently owns the seat over a small IPC protocol — reacting to VT-switch signals by revoking and reassigning them, so a compositor never has to implement VT arbitration itself, and never needs elevated privileges to begin with.

4.3 THREE IMPLEMENTATIONS, AND AN ABSTRACTION LAYER OVER THEM

IMPLEMENTATION	WHAT IT ACTUALLY IS
<code>systemd-logind</code>	Part of <code>systemd</code> proper. Exposes session/seat state over a D-Bus API (<code>org.freedesktop.login1</code>) that a lot of desktop software — not just compositors — queries directly: <code>polkit</code> 's authentication prompts, GNOME/KDE's own session management, suspend/inhibit locks all go through it. The default on most desktop distributions specifically because that D-Bus API is a de facto standard other software already expects.
<code>elogind</code>	A standalone extraction of <code>logind</code> — the same <code>org.freedesktop.login1</code> D-Bus API, the same session/seat/VT-switch behavior, packaged to build and run without the rest of <code>systemd</code> . Exists specifically so non- <code>systemd</code> distributions can still run desktop software that was written assuming <code>logind</code> 's D-Bus interface exists.
<code>seatd</code>	A smaller, purpose-built daemon: no D-Bus, no session-management API beyond the one job — a minimal socket protocol for “give me this device” / “you no longer have this device.” Its own documentation describes it as depending only on <code>libc</code> . This workspace's choice (§9, §10), made explicitly to avoid pulling in <code>logind</code> , <code>full udev</code> , <code>journald</code> , and <code>networkd</code> for one seat-management socket.

Compositors that want to support more than one of these without three separate code paths link against `libseat` instead of any daemon's protocol directly — a small client library with backends for both `seatd` and `logind` / `elogind`'s D-Bus API, so the same compositor binary works against whichever is actually running on a given system. Weston (§10) is built this way.

SEATD INSTEAD OF SYSTEMD-LOGIND — AN OPEN QUESTION, NOT A CLOSED ONE

Choosing `seatd` was deliberate, to avoid the rest of what a full `systemd` install brings in — but `seatd` alone provides none of `logind`'s D-Bus session-management API, only device arbitration. Whether that's sufficient once a real desktop session (COSMIC, §11) needs the broader session-management surface `polkit` and friends expect is still an open question this system hasn't had to answer yet.

05

USERLAND BASICS: COREUTILS, SHELLS, AND LOGGING IN

Every Unix system needs a basic set of file and text commands — `ls`, `cat`, `cp`, `mv`, `rm`, and so on. “Coreutils” is the generic name for that toolset, not one specific program, and there are several real answers to what actually provides it:

IMPLEMENTATION	WHAT IT IS
GNU coreutils	The implementation most desktop Linux distributions ship — one binary per command.

IMPLEMENTATION	WHAT IT IS
BusyBox	A much older answer aimed at tiny/embedded systems: a single small C binary that crams coreutils and a shell, <code>init</code> , <code>mount</code> , <code>ps</code> , and dozens of other tools into one multi-call executable, symlinked under every command name it implements.
utils	A from-scratch reimplementation of just coreutils, in Rust — one multi-call binary, the same symlink-per-applet shape as BusyBox, but scoped to coreutils alone rather than a whole system toolbox.

Whichever is chosen, the shell and login program are separate concerns — coreutils gets you `ls` and `cp`, not a shell to type them into or a login prompt to authenticate at.

5.1 THE MULTI-CALL BINARY TECHNIQUE

BusyBox and utils both work the same way: one compiled binary contains every applet’s code, and a symlink exists for every command name it supports — `/bin/ls`, `/bin/cp`, and so on — all pointing at that one file. When the kernel `exec()` s a program reached through a symlink, it resolves the symlink to find the actual file to run, but still passes the **path used to invoke it** (`/bin/ls`, not the multi-call binary’s own name) as `argv[0]`. The binary’s own `main` reads that back, strips it down to a bare command name, looks it up in an internal dispatch table, and jumps straight into that applet’s implementation — one process image, one `exec()`, no subprocess spawned to “really” run `ls`.

This is why the applet name in `argv[0]` has to be trustworthy for dispatch to work at all. Ordinarily it is — but it’s technically just a string the caller chose, not something the kernel guarantees matches reality, and at least one context on this system’s own build host turned out to supply an empty value instead of the expected path, which a naive “trust `argv[0]`” dispatch would silently mishandle. Some tools defend against `argv[0]` spoofing entirely by cross-checking it against `AT_EXECFN`, an auxiliary value the kernel itself provides at process startup recording the path it actually resolved and executed — a second, kernel-sourced source of truth for what name a program was really invoked under, independent of whatever the caller claimed.

5.2 SHELLS: WHAT ACTUALLY HAPPENS AFTER LOGIN

A shell is not special from the kernel’s point of view — it’s an ordinary program, distinguished only by what it does: read a line of text, parse it into a command and its arguments, `fork()` a child process, `exec()` the named command inside that child, wait for it to finish, and repeat. Running **interactively** (prompting at a terminal) and running as a **script interpreter** (reading commands from a file, no prompt) are the same binary in two different modes, not two different programs.

`sh` names the POSIX-standardized baseline shell language every Unix scripting relies on being available; `bash` is a strict superset of it — arrays, `[[ ]]` conditionals, command history, and more — and is commonly **also** what `/bin/sh` itself points at, making the extended dialect available under the POSIX name too (as opposed to systems that deliberately point `/bin/sh` at a smaller, POSIX-only implementation for scripts, keeping `bash` reserved for interactive login use). This system takes the former approach: `/bin/sh` is a plain symlink to `bash`.

5.3 AUTHENTICATION: WHAT LOGIN ACTUALLY CHECKS

Two files, both keyed by username, hold everything `login` needs:

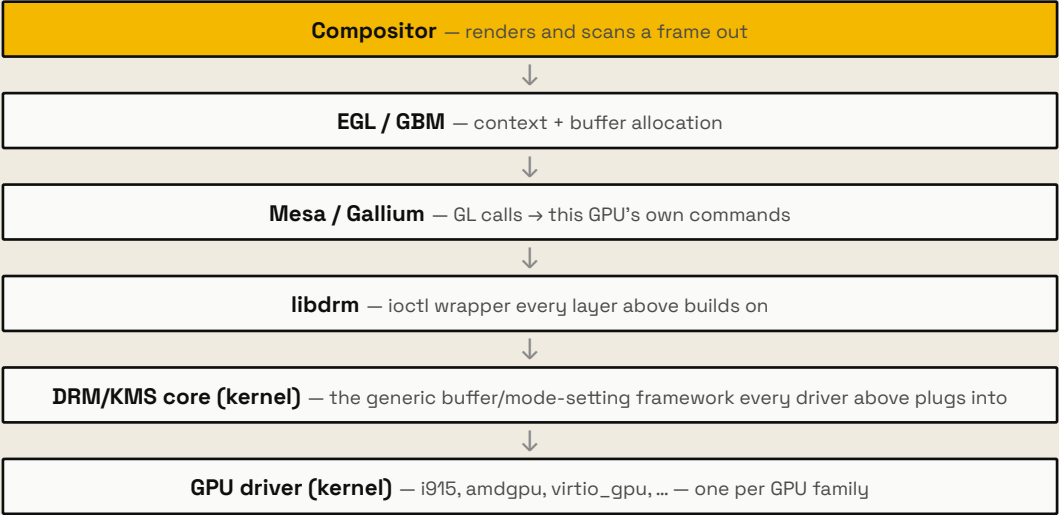
FILE	WHAT IT HOLDS
<code>/etc/passwd</code>	One line per account: username, a password placeholder, numeric UID/GID, a display name field, the account’s home directory, and its login shell. World-readable by design — plenty of software needs to look up a username or resolve a UID to a home directory.
<code>/etc/shadow</code>	The actual password data, split out specifically because <code>/etc/passwd</code> is world-readable: a salted password hash (or, notably, an empty field), and account-aging data — when the password was last changed, and its minimum/maximum age, warning period, and expiry. Readable only by root.

`login`’s job is simple to state: read a typed username, look it up in both files, and — if `/etc/shadow`’s password field for that account is empty — skip the password prompt entirely and let the login proceed unauthenticated (a deliberate no-password account, not a bug); otherwise hash whatever the user just typed using the same algorithm and salt recorded in that stored hash, and compare. Only on a match does it finish becoming that user’s session: set the process’s UID/GID to the account’s, make itself the session leader and attach the controlling terminal, `chdir` to the account’s home directory, and finally `exec()` the shell named in `/etc/passwd` — replacing itself entirely, not spawning it as a child. **PAM** (Pluggable Authentication Modules) is the layer many systems interpose here instead, so login can be extended with other authentication sources (fingerprint readers, LDAP, two-factor) without changing `login` itself — not used in this workspace; `login` here does the `passwd/shadow` check directly.

06

GRAPHICS, CONCEPTUALLY: FROM A KERNEL DRIVER TO A FRAME ON SCREEN

Getting from “a kernel that can talk to a GPU” to “a compositor rendering a client’s window on an actual display” passes through several distinct layers, each solving a different part of the problem:



LAYER	WHAT IT ACTUALLY IS
GPU driver (kernel)	The actual per-hardware driver — <code>i915</code> (Intel), <code>amdgpu</code> , <code>virtio_gpu</code> (for a virtual machine’s paravirtualized GPU), and so on. Implements DRM’s generic callbacks for one specific GPU family ; nothing above this layer talks to hardware registers directly.
DRM/KMS (kernel)	The generic subsystem every GPU driver plugs into: hands out and tracks buffers, submits command buffers, configures what’s actually scanned out to a display. Has no idea what “draw a triangle” means, and no hardware-specific code of its own — purely a resource-management, submission, and mode-setting framework .
Mesa	The userspace library that turns OpenGL/OpenGL ES calls into whatever a specific GPU actually understands.
Gallium	Mesa’s own internal plumbing for doing that translation once per GPU family rather than once per API. A “Gallium driver” is the translator for one specific GPU — or, for a virtual machine, one specific virtual GPU.
DRI	Direct Rendering Infrastructure — the convention by which an application actually finds and loads the right driver at runtime.
EGL	The glue between a window system (Wayland, X11, ...) and an OpenGL/GLES context. What a compositor or client actually links against directly — not Mesa’s internals.

LAYER	WHAT IT ACTUALLY IS
GBM	Generic Buffer Management — how a Wayland compositor allocates the actual pixel buffers it hands to the DRM/KMS display hardware.

6.1 DRM AND KMS: WHAT THE KERNEL ACTUALLY OWNS

DRM (Direct Rendering Manager) is a generic **framework**, not hardware code itself — the actual per-GPU driver (`i915`, `amdgpu`, `virtio_gpu`, ...) is a separate kernel module that implements DRM's callbacks for one specific GPU family; DRM/KMS core provides the buffer/mode-setting machinery every one of those drivers plugs into, uniformly, regardless of which GPU is actually underneath. DRM itself is really two jobs in one subsystem. The first, **GEM** (Graphics Execution Manager), is buffer-object management: every chunk of GPU-accessible memory — a texture, a framebuffer, a command buffer — is a GEM object, referenced by a handle the kernel hands back to whichever process allocated it, and GEM is what tracks who owns what and submits queued command buffers to the GPU in order. The second, **KMS** (Kernel Mode Setting), is display configuration: enumerating the physical **connectors** (an HDMI port, say), the **CRTCs** (the hardware pipeline that scans a buffer out to a connector at a given resolution/refresh rate), and the **planes** a CRTC can composite together — and letting userspace configure all of it through one atomic ioctl that either applies a whole new configuration or fails without touching anything, rather than a sequence of individual calls that could leave the display in a half-configured state partway through.

DRM exposes two different kinds of device node for this, deliberately separated by privilege: `/dev/dri/card0` (the **primary** node) is what mode-setting and display configuration happens through — only ever opened by whichever single process currently owns the display, the compositor. `/dev/dri/renderD128` (a **render** node) is for GPU work that has nothing to do with display configuration — submitting rendering or compute work, with no ability to touch what's actually on screen. A sandboxed or unprivileged client can safely be handed a render node; it can never be safely handed the primary node too.

6.2 BUFFER OBJECTS, AND SHARING THEM WITHOUT COPYING

Not every buffer needs GPU acceleration to produce — a plain, linear, CPU-writable framebuffer (a **dumb buffer**, in DRM's own terminology, created via `DRM_IOCTL_MODE_CREATE_DUMB`) is what a kernel framebuffer console uses, since it just needs somewhere to memcpy text glyphs into, not a GPU pipeline. Actual rendering — anything Mesa/Gallium produces — allocates real, driver-specific GEM buffer objects instead, through `libgbm` (below), sized and laid out however that particular GPU driver needs them internally.

Getting a buffer from “something Mesa just rendered into” to “something DRM/KMS scans out to a display” without a wasteful copy is `dma-buf`'s job: a kernel framework for exporting any GEM buffer as a plain file descriptor that a **different** subsystem (or process) can import and use directly — the same underlying memory, not a copy. This is the handoff that makes zero-copy presentation possible at all: a compositor renders into a buffer via EGL/Gallium, exports it as a `dma-buf` fd, and hands that exact fd to KMS to display, with the GPU's own rendered pixels never round-tripping through the CPU.

6.3 MESA, GALLIUM, AND FINDING THE RIGHT DRIVER AT RUNTIME

Mesa doesn't hard-code one driver into `libGL` / `libEGL` — which Gallium driver actually handles a given GPU is resolved at **runtime**, via **DRI**: the loader inspects which DRM device is being used (its PCI vendor/device ID, for real hardware, or its virtual device identity for `virtio-gpu`) and `dlopen()`s the matching driver shared object out of Mesa's own DRI driver directory. This is what lets one Mesa install support several different GPU families simultaneously without an application needing to know or care which one it's actually running against — including a Gallium driver for a device that only exists inside a virtual machine.

6.4 EGL AND GBM: FROM AN API CALL TO AN ALLOCATED BUFFER

EGL is what a client or compositor actually calls to get a usable OpenGL/GLES rendering context tied to a specific window-system surface — `eglCreateWindowSurface` and friends. It needs a concrete **platform** implementation to know what a “surface” even means for a given window system: `platform_wayland` for an ordinary Wayland client, and, for the compositor itself (which has no window system underneath it — it **is** the window system), `platform_gbm`. **GBM** is what backs that platform: `gbm_device` wraps a DRM file descriptor,

`gbm_surface / gbm_bo` allocate actual buffers sized and tiled correctly for that GPU's KMS scanout path, and `gbm_bo_get_fd()` is the call that exports one of those buffers as the `dma-buf` fd KMS ultimately consumes.

6.5 GETTING A RENDERED FRAME ONTO THE ACTUAL DISPLAY

The compositor's own per-frame loop, once everything above is in place, is genuinely simple: render the next frame into a GBM buffer via EGL, then hand that buffer to KMS as the next scanout buffer for a CRTC/plane via an atomic commit (conceptually a `drmModePageFlip`, though the modern atomic API folds this into the same all-or-nothing ioctl KMS configuration itself uses) — timed to the display's own vertical blank, not applied instantly mid-scan. Completion isn't polled for: the compositor reads its own DRM file descriptor as part of its normal event loop, and a page-flip-complete event arrives on it once the hardware has actually switched to scanning out the new buffer, which is also the compositor's cue that the **previous** buffer is now safe to reuse or free.

6.6 VIRTIO-GPU AND VIRGL: THIS SAME STACK, VIRTUALIZED

Everything above assumes a real GPU underneath, but QEMU doesn't present one when running as a plain virtual machine — `virtio-gpu` is a **paravirtualized** GPU device instead: the guest kernel's `virtio_gpu` DRM driver looks, from Mesa's side, like an ordinary DRM device (it still exposes GEM, KMS, render/primary nodes), but the actual rendering commands a guest's `virgl` Gallium driver produces are serialized and sent over the virtio transport to QEMU's own `virglrenderer`, which replays them against the **host's** real OpenGL driver and GPU. The guest never touches host GPU memory directly; it gets real GPU-accelerated rendering by proxy. `softpipe`, by contrast, is a genuine software rasterizer with no host GPU involved at all — pure CPU — and is what this workspace's graphics stack (§10) actually falls back to inside QEMU, `virgl`'s host-side `virglrenderer` support not being assumed present on every build/test host.

A minimal Wayland-only graphics stack built from source needs, at minimum: the kernel's DRM driver for the target GPU, `libdrm` (the kernel-userspace ioctl wrapper every one of these layers builds on), and Mesa itself providing `libEGL / libGLv2 / libgbm` plus at least one Gallium driver. `llvmpipe` (an LLVM-based software rasterizer) is one possible fallback driver, not a hard requirement — `virgl` and `softpipe` are two others that work without pulling LLVM into the build at all.

07 STATIC VS DYNAMIC LINKING

A binary can be built two ways: **statically linked**, where every library it needs is compiled directly into the one executable file, or **dynamically linked**, where the executable instead records which shared libraries (`.so` files) it needs and expects to find them on the target system at run time. Static linking means “compile once, copy one file, done” — no shared-library bookkeeping on the target at all — but it only scales to programs whose dependencies are genuinely small and self-contained. Anything that needs a library with real runtime state or plugin-style extensibility (D-Bus, a graphics driver stack) is effectively required to link dynamically, which means the target system needs those shared libraries physically present and findable by the dynamic linker before the program will run at all.

PART TWO

How distroless and distro each build one

08 THIS WORKSPACE

CRATE	ROLE
buildpack-core	Foundational library: the <code>Buildpack</code> trait every package implements, <code>DistroConfig</code> (the shared config-file format), the <code>PipelineStage</code> trait for whole-image operations, and the dependency-graph/build helpers both distros use.
buildpacks	One implementation per upstream package — kernel, coreutils, every library in §9–10's tables — shared between whichever distro(s) actually use each one.
builder-tui	A generic interactive terminal dashboard for running any stage of either distro's pipeline and writing the result to a USB stick.
distroless	musl + BusyBox + utils — a small, minimal distro. §9.
distro	A from-scratch glibc system, native-compiled, aimed eventually at a COSMIC desktop. §10.
distro-init	A from-scratch PID 1 written for this project, ~100 lines of Rust — used by <code>distro</code> (§10.1.1).

8.1 THE SHARED BUILDPACK ARCHITECTURE

Every upstream package this workspace builds — for either distro — is one self-contained value implementing the `Buildpack` trait: it declares its own source location, its dependencies on other buildpacks, how to fetch and build it, and what files it produces.

CONCEPT	WHAT IT DOES
Buildpack trait	<code>fetch()</code> / <code>build()</code> / <code>outputs()</code> / <code>clean()</code> plus a <code>describe()</code> for the TUI. One implementation per upstream package, in <code>buildpacks</code> .
DistroConfig	The shared config-file format both <code>distro.toml</code> and <code>distroless.toml</code> load through — <code>build_dir</code> , <code>networking</code> , and <code>image</code> are the only fields it understands directly; every other section is handed, unparsed, straight to that package's own buildpack.
Dependency graph	A real topological sort over every buildpack's declared dependencies decides build order — not a hand-maintained call sequence. Rendered fresh to an SVG (<code>dependency-graph.svg</code>) on every build.
PipelineStage trait	For the three operations that act on the whole assembled image rather than any one package: partitioning and writing the disk image, booting it in QEMU, and writing it to a USB stick. Identical logic for both distros.
CLI subcommands	<code>list-packages</code> , <code>fetch-pkg <id></code> , <code>build-pkg <id></code> , <code>clean-pkg <id></code> — inspect or rebuild one package at a time, on either distro.
builder-tui	An interactive dashboard driven by the same registry of buildpacks — shows build status per package, runs any stage, walks through settings (networking, kernel features, hostname), and writes to a USB stick with a confirmation prompt.

Each package's config lives under a `[section]` named after its buildpack id — a version and a source URL, at minimum. `distro.toml` and `distroless.toml` differ only in which sections they define, not in format.

09

HOW DISTROLESS BUILDS A LINUX SYSTEM

`distroless` is deliberately the smallest of the two: `musl libc`, `BusyBox` for the entire `coreutils/shell/init/` device-management job it can cover, plus `utils` and the kernel as the two things built from source beyond `BusyBox` itself. Everything in it is cross-compiled against `musl` rather than the host's own `glibc`.

9.1 BOOT SEQUENCE

STEP	COMPONENT	WHAT HAPPENS
1-2	GRUB, kernel	Same as §2's generic sequence.
3	BusyBox <code>init</code>	Reads <code>/etc/inittab</code> : runs <code>/etc/init.d/rcS</code> once at boot, then starts and respawns <code>getty</code> on the console.
4	<code>rcS</code>	A short shell script: mounts <code>/proc / /sys / devtmpfs</code> , sets the hostname, and — if networking is enabled — brings up <code>lo</code> and runs <code>udhcpc</code> for a DHCP lease.
5-7	<code>getty</code> → <code>login</code> → <code>shell</code>	BusyBox's own <code>getty / login / ash</code> — a single root account with no password set.

9.2 PACKAGES

PACKAGE	PROVIDES	NOTES
Linux kernel	<code>vmlinuz</code>	The same <code>Kernel</code> buildpack <code>distro</code> uses — kernel builds have no <code>musl/glibc</code> -specific behavior.
<code>utils/coreutils</code>	<code>ls</code> , <code>cat</code> , <code>cp</code> , ...	The <code>musl</code> variant of the same <code>Utils</code> buildpack <code>distro</code> uses (§10.3.1) — one multi-call binary, statically linked, <code>musl-static</code> by default.
BusyBox	<code>shell</code> , <code>init</code> , <code>mount</code> , <code>getty</code> , and the rest of a minimal system toolbox	Built via a curated <code>Kconfig</code> : <code>allnoconfig</code> , then a specific applet list enabled and cross-compiled against <code>musl</code> , statically linked.

9.3 PIPELINE

DISTROLESS'S CLI SURFACE

```

distroless fetch           # download + extract kernel, utils, busybox
distroless build-toolchain # install musl-gcc + the musl Rust target (once)
distroless build-kernel   # the kernel buildpack
distroless build-userland  # utils + busybox
distroless assemble-rootfs # merge it all into build/rootfs
distroless make-image      # partition + GRUB + write the disk image
distroless test-qemu [--window] # boot it
distroless write-usb --device <dev> # dd to real hardware (destructive)
distroless tui             # interactive dashboard for all of the above
distroless all             # the whole pipeline, in order

```

Assembling the `rootfs` copies `utils`' and `BusyBox`'s declared outputs in directly (`utils`' applet symlinks, `BusyBox`'s own `bin/sh`, `sbin/init`, and applet `set`), installs the kernel modules this build produced, and writes the handful of config files `BusyBox`'s `init` expects (`/etc/inittab`, `/etc/passwd`, `/etc/fstab`, `/etc/init.d/rcS`, and — when networking is enabled — a `udhcpc` `bound/renew/deconfig` script).

VERIFIED
A scripted QEMU boot reaches the login prompt, logs in via BusyBox `ash` with no password, and confirms `ls /bin | wc -l` and a correct `uname -a` against the just-built kernel.

10

HOW DISTRO BUILDS A LINUX SYSTEM

`distro` is the larger of the two: a from-scratch **glibc** system, native-compiled against the host’s own tool-chain — no cross-compilation — aimed eventually at running a COSMIC desktop. Where `distroless` reaches for one BusyBox binary to cover `coreutils`, `shell`, `init`, and device management all at once, `distro` builds a separate, purpose-specific package for each of those jobs, plus (currently) the full graphics stack needed to host a Wayland compositor.

10.1 BOOT SEQUENCE

STEP	COMPONENT	WHAT HAPPENS
1-2	GRUB, kernel	Same as §2’s generic sequence.
3	distro-init	Our own PID 1 (below). Mounts <code>/proc</code> , <code>/sys</code> , <code>/dev</code> (<code>devtmpfs</code>), and a fresh <code>tmpfs</code> at <code>/run</code> .
4	udev, systemd, dbus-daemon	Forked and exec’d, in that order. <code>distro-init</code> then runs <code>udevadm trigger</code> once — a coldplug that tells <code>udev</code> about the devices <code>devtmpfs</code> already created before it started.
5	agetty x2	Forked on <code>ttys1</code> (VGA console) and <code>ttys0</code> (serial, for QEMU/headless use). Whichever exits gets respawned.
6	login	<code>agetty</code> execs <code>/bin/login</code> once a username is typed. Reads <code>/etc/passwd</code> / <code>/etc/shadow</code> for real — the <code>root</code> account’s shadow entry has an empty password field, so <code>login</code> skips the password prompt.
7	bash	<code>login</code> execs the shell named in <code>/etc/passwd</code> .

VERIFIED LIVE
Every arrow above was reproduced in QEMU with a scripted serial-console session: `login: root` → no password prompt → `-bash-5.2#` → `dbus-send --system ... ListNames` returns a real reply → `udevadm info` returns a populated device entry.
`--query=all --name=/dev/tty1`

distro-init
Nothing **glibc**-based has a BusyBox-equivalent single `init` binary, and pulling in `systemd` would drag in `logind`, `udev`, `journal`, and far more than this system currently needs. `distro-init` is a purpose-built `init` instead — about 100 lines of Rust using the `nix` crate — following exactly the “purpose-written `init`” pattern described in §2.5.

DISTRO-INIT/SRC/MAIN.RS — THE WHOLE SUPERVISION LOOP, ABBREVIATED

```
fn main() {
    mount_basic_filesystems();           // proc, sysfs, devtmpfs, tmpfs at /run

    let mut udevd_pid = spawn_udev();    // no -d: stays a tracked child, not a daemon
    let mut seatd_pid = spawn_seatd();
    let mut dbus_pid = spawn_dbus();     // --nofork, same reasoning
    coldplug_devices();                  // `udevadm trigger`, one-shot

    let mut ttyl_pid = spawn_ttyl();     //agetty
    let mut serial_pid = spawn_serial();

    loop {
        match waitpid(None, Some(WaitPidFlag::empty())) {
            Ok(WaitStatus::Exited(pid, _)) | Ok(WaitStatus::Signaled(pid, _, _)) => {
                // whichever of the five died gets forked again; anything
                // else reaped here was just an orphaned grandchild.
            }
            ...
        }
    }
}
```

Every daemon it starts would normally daemonize itself (fork, detach, exit the parent) — `dbus-daemon --nofork` and `udev` with `no -d` suppress that, so they stay `distro-init`'s direct children and its `waitpid` loop actually sees them exit if they crash.

10.2 THE KERNEL

`distro.toml` names a kernel version and a download URL under `[kernel]`; `distro build-kernel` fetches, configures, and compiles it.

Kconfig and .config : how kernel configuration actually works

Every buildable kernel feature — a driver, a subsystem, an optional instrumentation hook — is declared in **Kconfig**, a dependency-aware description language: an option can be built (`y` , directly into the kernel image), built as a separate loadable module (`m`), or left out entirely (`n`), and Kconfig itself enforces that an option can't be turned on if whatever it depends on is off. `.config` is the flat, resolved output of that — one `CONFIG_FOO=y / m / n` line per option — and it's `.config`, not the Kconfig source tree, that `make` actually reads to decide what to compile.

`make defconfig` produces a **curated** starting `.config` — each architecture ships its own reasonable, working baseline, not “every option off” — which is why stripping this project's own optional feature packs back out (below) starts from a real, bootable configuration rather than building one up from nothing. `make olddefconfig` is the companion operation for the opposite situation: given an existing `.config` (possibly

written by a different Kconfig tree version entirely — a newer kernel adds options an older `.config` never had opinions about), it fills in every option the current tree knows about but the file doesn't mention with that option's own default, non-interactively. This project uses it in both of §10.2's starting points: after `defconfig`'s own baseline gets edited by `pack stripping`, and to re-resolve a previously saved, hand-picked config that may predate a kernel version upgrade.

Built-in vs. module: why nothing here needs loading

An option built as `y` is simply part of `vmlinuz` — present the moment the kernel starts, nothing further to do. An option built as `m` instead produces a separate `.ko` file, meant to be found under `/lib/modules/<version>/` and explicitly loaded (by an early-boot module loader, or a later `modprobe`) before whatever it provides becomes available — a real mechanism, and the right choice for a general-purpose distribution that can't predict every piece of hardware it'll run on ahead of time. This distro's own kernel takes the other path: every feature it builds is `y`, never `m` — no `/lib/modules` install step exists in its own pipeline (§10.5) at all — which is exactly what makes skipping an `initramfs` (§2.3) safe: there's no module-loading step this kernel could ever be waiting on before it can mount its own root filesystem.

Feature packs, and getting from a stripped baseline to the final config

The starting config is either `make defconfig` (the default) with every optional **feature pack** below stripped back off, or a previously saved `.config` from an interactive `make menuconfig` session (`distro menu-config`), re-resolved via `make olddefconfig`. Either way, whatever packs are named in `kernel.features` get turned back on as the final step.



PACK	WHAT IT TURNS ON
graphics	DRM/KMS graphics + fbdev console (i915, virtio-gpu, bochs, AGP) instead of plain VGA text.
sound	ALSA sound subsystem and the Intel HDA driver.
wireless	The Wi-Fi stack (cfg80211/mac80211) and rkill.
hid-extras	Per-vendor HID quirk drivers and the hiddev/hidraw userspace interfaces — generic USB HID keyboards/mice work without this.
legacy-nics	Dedicated Ethernet chipset drivers for real hardware — QEMU's virtio-net always works without this.
legacy-buses	PCMCIA/CardBus and legacy PATA chipset drivers.
network-fs	NFS (client + root-over-NFS), 9P, and autofs.
netfilter	Connection tracking, NAT, and iptables.
security-extras	Disk quotas, POSIX ACLs, and SELinux.
iommu	AMD/Intel IOMMU support — PCI passthrough or virtualization-host use.
debug	Kernel debug instrumentation — useful while bringing up boot, dead weight once stable.
ia32-emulation	Run 32-bit x86 binaries on this 64-bit kernel.
iso9660	ISO9660/Joliet/zisofs filesystem support.
boot-logo	Framebuffer console + boot-time logo (stock penguin, or a custom 80×80 PPM via <code>kernel.logo_file</code>).

`distro.toml` currently sets no `kernel.features` at all — the kernel ships the fully stripped `defconfig` baseline plus whatever `graphics` support the rootfs actually needs is enabled directly, not via a named pack.

10.3 PACKAGES

`distro` builds a separate, purpose-specific package for each job a desktop-capable system needs done — laid out here in dependency order, bottom of the stack first: the static base first (needs nothing else already built), then the seat/session layer, then the Wayland-core libraries, then the graphics stack proper, then everything Weston itself needs.

The static base

Five packages exist purely to get from a mounted rootfs to a real, authenticated shell — the glibc equivalent of what BusyBox does as one binary, spread across separate upstream projects because glibc-land has no single equivalent to reach for. All five are statically linked: one binary compiled, one binary copied into the rootfs, nothing else to track.

PACKAGE	VERSION	PROVIDES	WHY IT'S THERE
utils/coreutils	git main	ls, cat, cp, ...	Rust reimplementations of GNU coreutils, chosen over BusyBox for this distro specifically to prefer Rust alternatives where practical.
bash	5.2.37	/bin/bash, /bin/sh	The login shell named in <code>/etc/passwd</code> .
util-linux	2.41.2	agetty, mount, umount	Built with only these three programs enabled — util-linux ships dozens, only these are needed here.

PACKAGE	VERSION	PROVIDES	WHY IT'S THERE
shadow-utils	4.17.4	login, passwd	Real /etc/passwd + /etc/shadow authentication.

Seat & session daemons

Three daemons do what a desktop session needs before any compositor can run — none of them optional, the floor a compositor stands on rather than decoration:

PACKAGE	VERSION	WHAT IT ACTUALLY DOES
seatd	0.9.3	Owns /dev/input/* and /dev/dri/* on behalf of whatever process asks over its socket — the seat-management daemon described conceptually in §4, chosen over systemd-logind to avoid pulling in the rest of systemd.
dbus	1.16.2	The system message bus. Runs as root here — the rootfs has no unprivileged messagebus user yet to drop privileges to.
udev	3.2.14	The systemd-independent udev fork described in §3. Exists in this pipeline specifically because libinput hard-depends on libudev.

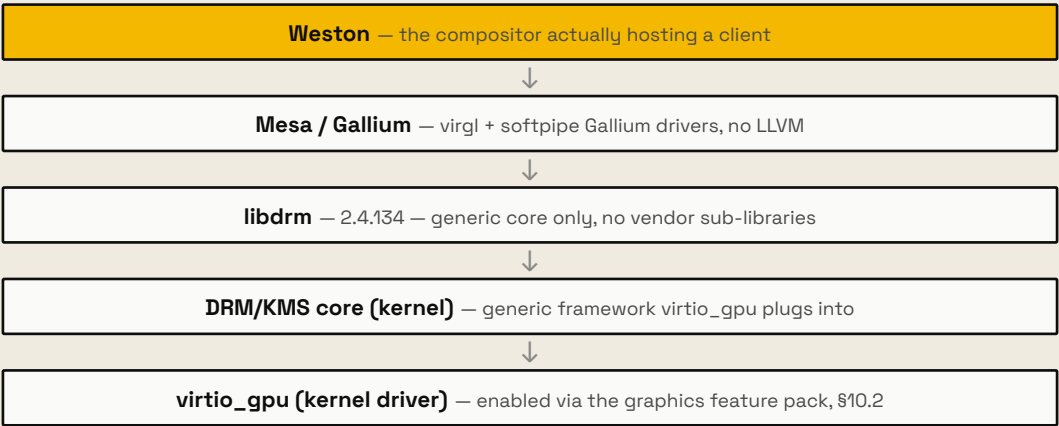
The Wayland-core libraries

Seven libraries sit ready for a compositor to link against — nothing in this group runs on its own; verification at this layer is “builds and installs cleanly,” not “does something observable”:

PACKAGE	VERSION	WHAT IT'S FOR
wayland	1.26.0	The core wire-protocol libraries and wayland-scanner, the code generator every later Wayland package needs at its own build time.
wayland-protocols	1.49	The actual protocol definitions — xdg-shell and the rest — as XML.
libxkbcommon	1.12.4	Turns “us, evdev, pc105” into the keymap tables a compositor hands to clients.
pixman	0.46.4	Software rasterization — Mesa’s fallback path.
libdisplay-info	0.4.0	Parses a monitor’s own EDID/DisplayID.
libevdev	1.13.7	Reads and writes raw evdev input-device events — libinput’s one mandatory dependency.
libinput	1.31.3	Turns raw evdev events into pointer/keyboard/touch/gesture events a compositor actually wants.

The graphics stack

From §6’s conceptual layering down to what’s actually built here — the one place in this whole table where a **kernel-side** driver matters as much as anything built from userspace source, since nothing above it works without the right one already compiled in via kernel.features (§10.2.3):



PACKAGE	VERSION	WHAT IT'S FOR
libdrm	2.4.134	The kernel-userspace ioctl wrapper every GPU-facing library builds on — every vendor-specific sub-library disabled, virtio-gpu needs only the generic core.
Mesa	26.2.2	libEGL, libGL, libGLESv2, libgbm, and the Gallium driver — built scoped to virgl (talks to QEMU's virtio-gpu/virgl backend) and softpipe (software fallback). Vulkan and GLX/X11 both disabled — Wayland/EGL/GLES only, no LLVM needed for either driver.

The kernel driver itself — `virtio_gpu`, part of the `graphics` feature pack (§10.2.3, `CONFIG_DRM_VIRTIO_GPU`) — is not a separate buildpack: it's compiled directly into `vmlinuz` alongside the rest of the kernel, the same `y -not- m` choice §10.2.2 covers for every other driver this kernel needs. Nothing in `libdrm`/Mesa's own build depends on which GPU driver ends up underneath at runtime — that binding happens at boot, when the kernel probes for a matching device and the driver it finds (`virtio-gpu`, here, since that's what QEMU presents) is whatever DRI then loads a Gallium driver against (§6.3).

Hosting a compositor: the Weston chain

Hosting an actual compositor needs six more packages, plus a keyboard layout data package:

PACKAGE	VERSION	WHAT IT'S FOR
zlib	1.3.2	Compression — libpng's and cairo's own dependency.
expat	2.7.1	XML parsing — fontconfig's config-file dependency.
libpng	1.6.44	PNG images — Weston's own unconditional dependency, and cairo's PNG surface backend.
FreeType	2.13.3	Font rasterization.
fontconfig	2.15.0	Font matching — FreeType's companion, needed for cairo's font backend.
cairo	1.18.2	2D graphics — Weston's mandatory window-decoration dependency. Image-surface only, no X11/xcb/GL backends.
xkeyboard-config	2.44	Keyboard layout/rules data — no code, what <code>libxkbcommon</code> looks for at runtime.
Weston	16.0.0	The compositor itself — DRM backend, GL renderer, kiosk shell, <code>weston-simple-egl</code> . Vulkan/X11/Xwayland/systemd/JPEG/WebP/LCMS2 all disabled.

VERIFIED — THE ACTUAL MILESTONE

A real QEMU boot: Weston (launched with a minimal `weston.ini` selecting its kiosk shell) initializes DRM, EGL/GL via this workspace's own Mesa `softpipe` Gallium driver, `libinput`, and the kiosk shell, and opens a Wayland socket. `weston-simple-egl`, run against that socket, connects and renders without error — a minimal Wayland client actually rendering via `virtio-gpu` inside QEMU. Not yet automatic at boot: today Weston is started by hand from a login shell, not a `distro-init` service.

10.4 THE SYSROOT: HOW THESE PACKAGES FIND EACH OTHER

Static-base packages (§10.3.1) never need each other at build time — each just needs the host's `gcc`. Everything from the seat layer onward does: `wayland-protocols` needs `wayland-scanner` on `PATH` at its own build time, and `libinput` needs `eudev`'s installed `libudev.pc` to link against `libudev` at all. Solving that — letting one from-source package's build see another already-built one, using the exact same mechanism real software expects a real system install to provide — is what the **sysroot** actually is.

```
Package A: --prefix=/usr → make install DESTDIR=sysroot → .pc file in sysroot → Package B's pkg-config lookup → Package B build
```

DESTDIR: staged installation, not a different install

`./configure --prefix=/usr` (or `meson's --prefix=/usr`) doesn't just control where `make install` copies files to — it's compiled directly into the resulting binaries, since plenty of software looks up its own data files, plugins, or config at **runtime** relative to whatever prefix it believes it lives under. Actually installing to `/usr` while building would mean every package installs on top of the host's own real `/usr` —

exactly what this project’s whole premise rules out. `DESTDIR` is the standard convention (every autotools/meson project supports it, because real distribution packaging depends on it) for breaking that link: `make install DESTDIR=<sysroot>` still installs **as if** the package’s prefix were `/usr`, but physically writes every file under `<sysroot>/usr/...` instead — the binary’s own compiled-in idea of its prefix stays the real `/usr`, only the location `make install` actually wrote to changes. This is precisely how `.deb` / `.rpm` build pipelines stage a package’s files before archiving them, reused here for the same reason: to get a package’s files onto disk without overwriting the host.

pkg-config: how one package’s build finds another’s

`pkg-config` itself is not one of this distro’s own from-source packages — like `gcc`, `meson`, `ninja`, and `gperf`, it’s a host build tool `distro build-toolchain` installs via `apt-get` (`distro/src/stages/toolchain.rs`), the same “host compiler toolchain is infrastructure, not distro content” exception every from-scratch distro makes for the tools that do the actual compiling. What follows is how the packages **it operates on** — the ones this distro does build from source — use it to find each other.

Every one of these packages exposes what it provides to later builds via a `.pc` file — a plain text file recording its compiler/linker flags (`Cflags`, `Libs`) under whatever prefix it was configured with. A later package’s build runs `pkg-config --cflags --libs libfoo` and gets back exactly those flags, with no need to know `libfoo`’s install layout itself. Two environment variables make this work against a `sysroot` instead of a real system install: `PKG_CONFIG_PATH` points `pkg-config` at the `sysroot`’s own `.pc` file directory instead of the host’s, and `PKG_CONFIG_SYSROOT_DIR` rewrites the `-I` / `-L` paths those `.pc` files record — written as if `/usr/...` were real — into the `sysroot`’s actual, on-disk `<sysroot>/usr/...`, since the `.pc` files themselves were generated assuming a real install, exactly per the `DESTDIR` convention above.

wayland’s one exception, on purpose

One package in this chain is deliberately built **without** `DESTDIR` at all: `wayland` itself is configured with a real, absolute `--prefix=<sysroot>/usr` and installed directly. `wayland-scanner`’s own path is recorded in `wayland`’s `.pc` file as a **custom** `pkg-config` variable (`wayland_scanner=${bindir}/wayland-scanner`) — and unlike the ordinary `Cflags` / `Libs` fields, `PKG_CONFIG_SYSROOT_DIR` does not rewrite custom variables, because `pkg-config` has no way to know a given custom variable even represents a path. `wayland-protocols`’ build reads that variable and runs whatever it names, literally, as part of its own build — with a plain `--prefix=/usr` and `DESTDIR`, that would resolve to `/usr/bin/wayland-scanner`, a path that exists nowhere on the actual build host. Building `wayland` against its own real, final `sysroot` path instead sidesteps the problem entirely: this is safe specifically **because** none of `wayland`’s own shared libraries do a prefix-derived runtime lookup the way some of these packages’ daemons do — its `.so`s don’t care what prefix they think they’re under, only the one `wayland-scanner` invocation during a later package’s build does.

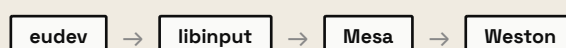
Dependency order: what has to build before what

None of this — `DESTDIR`, `PKG_CONFIG_SYSROOT_DIR`, `wayland`’s own exception — works if a package builds before whatever it needs is already sitting in the `sysroot`. With around twenty packages needing each other in varying combinations, that order isn’t hand-sequenced: each buildpack declares its own dependencies, and a real topological sort (§8.1) computes an order that satisfies every one of them, re-rendered to `dependency-graph.svg` on every build (§8.1) — what follows is a couple of representative edges out of that full graph, not the graph in full.

Mesa is the deepest single node in it — it can’t build until **six** other packages already have:

PACKAGE	DEPENDS ON
Mesa	<code>libdrm</code> , <code>wayland</code> , <code>libxkbcommon</code> , <code>pixman</code> , <code>libdisplay-info</code> , <code>libinput</code>
Weston	Mesa (via <code>libdrm</code> / <code>EGL</code>), <code>libinput</code> , <code>wayland</code> , <code>wayland-protocols</code> , <code>libxkbcommon</code> , <code>cairo</code> , <code>xkeyboard-config</code>

One concrete path through the graph, start to finish — illustrative, not exhaustive; `libinput` alone also needs `libevdev`, omitted here to keep the chain readable:



`eudev` has no `sysroot` dependencies of its own (§10.3.2 covers why it's needed at all — `libinput`'s hard `libudev` dependency), which is what lets it build first; everything downstream of it in this particular chain literally cannot start until `eudev`'s own `.pc` file exists in the `sysroot` for `PKG_CONFIG_SYSROOT_DIR` to find. Assembling the rootfs copies this whole `sysroot` tree in with one `cp -a` — every package that built into it, regardless of which mechanism (`DESTDIR` or a real `sysroot` prefix) actually put it there.

10.5 PIPELINE

DISTRO'S CLI SURFACE

```
distro fetch           # download + extract every source
distro build-toolchain # apt-get the host build tools (once)
distro build-kernel    # the kernel buildpack
distro build-userland   # every other package
distro assemble-rootfs # merge it all into build-distro/rootfs
distro make-image       # partition + GRUB + write the disk image
distro test-qemu [--window] # boot it
distro write-usb --device <dev> # dd to real hardware (destructive)
distro tui              # interactive dashboard for all of the above
distro all              # the whole pipeline, in order
```

`build-userland` fetches and builds every non-kernel package in dependency order (§8.1's topological sort); `assemble-rootfs` then copies every statically-linked package's declared outputs in directly (`coreutils` and its applet symlinks, `bash` + `bin/sh`, `util-linux`'s `agetty` / `mount` / `umount`, `shadow`'s `login` / `passwd`, `distro-init` as `/sbin/init`), bulk-copies the whole `sysroot` for every dynamically-linked package in one `cp -a`, then adds the host's own C library, `libstdc++`, and the handful of other runtime `.so`s these dynamically-linked packages need, plus the dynamic linker itself and `/etc/passwd` + `/etc/shadow`.

VERIFIED

A full pipeline run (`fetch` → `build-kernel` → `build-userland` → `assemble-rootfs` → `make-image`) reports every package “already exists” on a rebuild, and a full QEMU regression boot — `login`, `dbus-send`, `udevadm`, and Weston's own DRM/EGL/GL initialization — passes end to end.

11 WHAT ISN'T PART OF THE PICTURE YET

NEXT `rustup` / `cargo` on-target, plus a curated Rust-CLI-tools suite (`ripgrep`, `bat`, `eza`, ...) — no new architecture needed, each is just another buildpack.

THEN COSMIC itself — `cosmic-comp`, `cosmic-session`, `cosmic-panel`, `cosmic-greeter`, minimal subset first.

LATER More COSMIC components, real GPU drivers beyond `virtio-gpu`, audio, networking UI.

A STANDING RULE FOR EVERYTHING ABOVE

Anything that ends up in the built image — a buildpack's runtime output, a daemon, a COSMIC component, any future app — must be Rust or C. No Python, Ruby, Perl, or Node runtime ships on target. This is the same reasoning §10.3.1 already gives for choosing `utils` over `BusyBox`, made explicit as a constraint on every package picked from here on — network stack, audio stack, and the COSMIC components above included. Host-side build tooling is exempt: `meson` (Python) is infrastructure, not distro content, the same carve-out as `gcc` / `ninja` / `pkg-config` (§10.4.2) — only a package's **runtime** language is constrained, never its build system.

And three deliberate gaps in what's already built, worth knowing about rather than discovering later:

- `utils`' built feature set does not include `grep` or `sed` — they were never part of `coreutils`' scope upstream in the first place. The applet symlinks exist but dangle. Not yet fixed.
- `libxcb` was never built. Its only real consumer on a Wayland-native target is XWayland compatibility, not currently planned.
- `mtdev` (legacy multitouch) and `libwacom` (tablet identification) were both left out of `libinput` — niche hardware support, easy to add back later if a real device needs it.